Efficient Fine-Tuning of Llama 2 with QLoRA

1. Project Overview

In this project, we are fine-tuning a lightweight large language model. A pretrained Llama 2 chat model from Hugging Face is used, QLoRA for memory-efficient adaptation is applied, and the model is trained on a small instruction dataset.

This project serves as a proof-of-concept for parameter-efficient supervised fine-tuning. It demonstrates how to combine quantization, LoRA adapters, and the TRL SFTTrainer in a compact workflow that can be executed on limited hardware.

2. Problem Statement

Full fine-tuning of multi-billion-parameter language models is expensive in memory and compute. For a 7B chat model, loading and updating all weights in standard precision can exceed the capacity of a modest GPU. This project addresses this constraint by using a quantized base model and training only small LoRA adapter weights instead of the full parameter set.

- Reduce the memory footprint enough to make training feasible on a single-GPU style setup.
- Preserve the base model while adapting it to an instruction-style dataset.

3. Goals and Objectives

- Load a pretrained Llama 2 chat model and a compatible instruction dataset from Hugging Face.
- Configure QLoRA using 4-bit quantization and LoRA hyperparameters.
- Train the model with supervised fine-tuning using TRL's SFTTrainer.
- Log training activity to TensorBoard for monitoring.
- Save the trained artifact and run a post-training generation prompt as a smoke test.

The notebook achieves all five of these objectives in a linear sequence without additional orchestration code.

4. System Architecture and details of the model and dataset

The implemented architecture can be described as a six-stage pipeline:

Stage	Implemented Component	Purpose
1	Environment setup	Install Accelerate, PEFT, BitsAndBytes, Transformers, and TRL.
2	Configuration block	Define model, dataset, LoRA, quantization, optimizer, and training settings.
3	Data and model loading	Load the dataset, tokenizer, and quantized base model onto GPU 0.
4	Adapter attachment	Attach LoRA configuration for causal language modeling.
5	Training and logging	Fine-tune through SFTTrainer and send logs to TensorBoard.
6	Persistence and inference	Save the trained artifact and run a prompt through a text-generation pipeline.

Description of the models and dataset used in this project:

Categories	Description
Project Type	LLM fine-tuning workflow
Base Model	NousResearch/Llama-2-7b-chat-hf
Dataset	mlabonne/guanaco-llama2-1k
Training Approach	QLoRA: 4-bit quantization, LoRA adapters, supervised fine-tuning

High-Level Flow of the project pipeline:

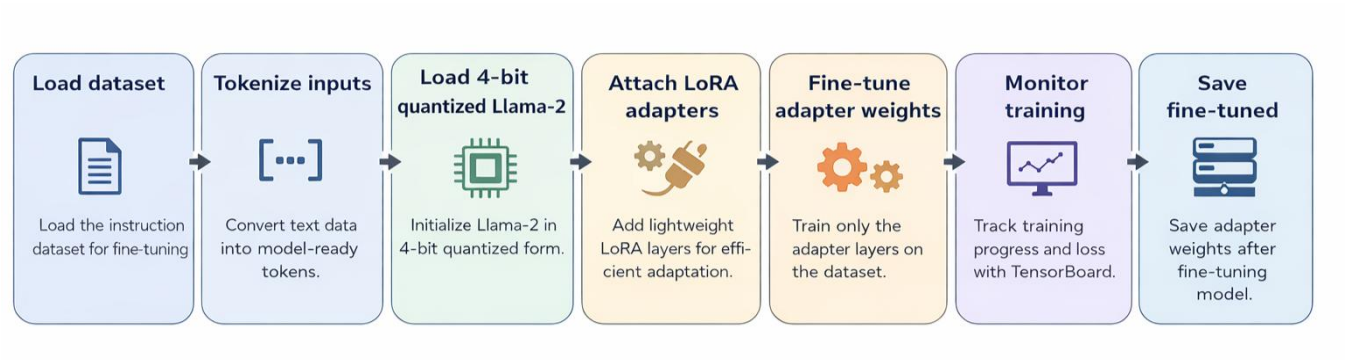

Figure 1. End-to-end fine-tuning flow.

5. Tools and Technologies Used

Tool / Library	Role in the Project
Python / Jupyter	Notebook execution environment for the full workflow.
Transformers	Loads the tokenizer, model, training arguments, and inference pipeline.
Datasets	Pulls the training split from Hugging Face datasets.
BitsAndBytes	Provides 4-bit quantization support for QLoRA.
PEFT	Defines the LoRA adapter configuration.
TRL	Runs supervised fine-tuning with a high-level trainer.
PyTorch	Supplies tensor types and GPU capability checks.
TensorBoard	Monitors training logs visually.

6. Methodology

This project follows a direct, incremental development process. All major settings are centralized first, then the training pipeline is assembled from the bottom up. This structure makes the experiment easy to understand and

modify because model configuration, quantization options, LoRA parameters, and training settings are all visible in one place before training starts.

6.1 Configuration Methodology

The configuration block is the control center of the notebook. It explicitly captures the choices that make this a QLoRA workflow: LoRA rank and scaling, 4-bit NF4 quantization, gradient checkpointing, paged AdamW optimization, cosine learning rate scheduling, and TensorBoard reporting.

```
model_name = "NousResearch/Llama-2-7b-chat-hf"
dataset_name = "mlabonne/guanaco-llama2-1k"
new_model = "Llama-2-7b-chat-finetune"

lora_r = 64
lora_alpha = 16
lora_dropout = 0.1

use_4bit = True
bnb_4bit_compute_dtype = "float16"
bnb_4bit_quant_type = "nf4"
use_nested_quant = False

num_train_epochs = 1
per_device_train_batch_size = 4
gradient_checkpointing = True
learning_rate = 2e-4
optim = "paged_adamw_32bit"
```

6.2 Model Loading and Quantization Methodology

The model is loaded with a BitsAndBytesConfig that activates 4-bit inference-time storage for the base weights. This is the key mechanism that cuts memory usage. The code also checks the CUDA device capability to suggest bf16 acceleration when the hardware is compatible, although bf16 is not enabled in the current configuration.

```
dataset = load_dataset(dataset_name, split="train")

bnb_config = BitsAndBytesConfig(
    load_in_4bit=use_4bit,
    bnb_4bit_quant_type=bnb_4bit_quant_type,
    bnb_4bit_compute_dtype=compute_dtype,
    bnb_4bit_use_double_quant=use_nested_quant,
)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map=device_map
)

tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)
tokenizer.pad_token = tokenizer.eos_token
```

6.3 Training Methodology

Training is performed with TRL's SFTTrainer. The notebook passes the dataset, tokenizer, LoRA configuration, and a TrainingArguments object to the trainer. The run is configured for one epoch, batch size four, gradient checkpointing, no example packing, and TensorBoard reporting. Because only LoRA adapters are trained, the notebook keeps the workflow compact and resource-aware.

```
peft_config = LoraConfig(
    lora_alpha=lora_alpha,
    lora_dropout=lora_dropout,
    r=lora_r,
    bias="none",
    task_type="CAUSAL_LM",
)

training_arguments = TrainingArguments(
    output_dir=output_dir,
    num_train_epochs=num_train_epochs,
    per_device_train_batch_size=per_device_train_batch_size,
    gradient_accumulation_steps=gradient_accumulation_steps,
    optim=optim,
    logging_steps=logging_steps,
    learning_rate=learning_rate,
    weight_decay=weight_decay,
    max_grad_norm=max_grad_norm,
    warmup_ratio=warmup_ratio,
    lr_scheduler_type=lr_scheduler_type,
    report_to="tensorboard"
)

trainer = SFTTrainer(
    model=model,
    train_dataset=dataset,
    peft_config=peft_config,
    dataset_text_field="text",
    tokenizer=tokenizer,
    args=training_arguments,
    packing=packing,
)
trainer.train()
```

7. Key Features Implemented

- Pretrained chat model loading from Hugging Face.
- QLoRA with 4-bit NF4 quantization.
- LoRA adapter-based fine-tuning for causal language modeling.
- Centralized hyperparameter configuration block.
- TensorBoard logging for training visibility.

7.1 Monitoring, and Inference Snippet

```
trainer.model.save_pretrained(new_model)

%load_ext tensorboard
%tensorboard --logdir results/runs

prompt = "What is a large language model?"
pipe = pipeline(task="text-generation", model=model, tokenizer=tokenizer, max_length=200)
result = pipe(f"<s>[INST] {prompt} [/INST]")
print(result[0]['generated_text'])
```

8. Solutions and Learnings

- QLoRA is a practical compromise between model adaptability and GPU limits.
- Centralizing hyperparameters makes experiment changes faster and easier to reason about.
- SFTTrainer reduces boilerplate and helps assemble a training run quickly.
- TensorBoard integration is valuable even in a short proof-of-concept because it provides visibility into the run.
- A prompt test is useful as a smoke test, but further LLM evaluation and benchmarks are always important to determine the value.

A key learning from this project is that resource-efficient fine-tuning is the interaction of several design choices: quantized loading, adapter training, memory-aware optimization, and restrained experiment scope.

9. Future Improvements

- Add a validation and test pipeline instead of relying only on training logs and a single prompt check.
- Record and report quantitative metrics such as loss curves, perplexity, or task-specific evaluation scores.
- Experiment with different LoRA ranks, dropout settings, and learning rates.
- Evaluate packing, checkpoint saving frequency, and multi-epoch training for better efficiency and recovery.